SKILL - 18

Name: k.vaishali

Rollno: 2520030117

Section: 8

Aim

To analyze signal delivery, understand signal masks, review process groups, explore terminal signals, and test signal behavior. Also, to implement recovery of stopped background jobs by sending continue signals, updating job status, maintaining the job table, verifying execution, and testing background recovery.

Objective

- To understand how signals are delivered to processes.
- To understand signal masks and blocked signals.
- To review the role of process groups.
- To explore terminal-generated signals.
- To test signal behavior with background jobs.
- To resume a stopped background job.
- To send a continue signal to a stopped process.
- To update job status after signal delivery.
- To maintain a valid job table.

.

Problem Statement

Design and implement a shell-style signal and background-job manager that can observe signals affecting processes, maintain information about process groups and job states, detect stopped jobs, send a continue signal to resume them, update their status, and verify that the recovered background job continues execution correctly.

Theory

1. Signal Delivery

A signal can be generated by the terminal, the operating system, or another process. The signal is delivered to a process or process group identified by a process ID or process-group ID.

2. Signal Masks

A process can temporarily block selected signals using a signal mask. Blocked signals are not delivered

immediately and may remain pending until the process unblocks them.

3. Process Groups

A process group is a collection of related processes with a common process-group ID. Terminal-generated signals such as interrupt or stop signals can be directed to the foreground process group.

4. Terminal Signals

Interactive terminals can generate signals in response to special key combinations. For example, Ctrl+C commonly generates SIGINT and Ctrl+Z commonly generates SIGTSTP, which stops the foreground process or process group.

5. Stopped Jobs

A job may enter the STOPPED state after receiving a stop signal. The shell must retain the job information so that the process can later be resumed instead of treating the job as completed.

6. Continue Signals

SIGCONT requests that a stopped process continue execution. After the process resumes, the shell updates the job state from STOPPED to RUNNING.

7. Job Status Updates

The shell monitors child processes and records whether a job is RUNNING, STOPPED, or COMPLETED. Status changes are detected through waitpid() and related process-status macros.

8. Background Recovery

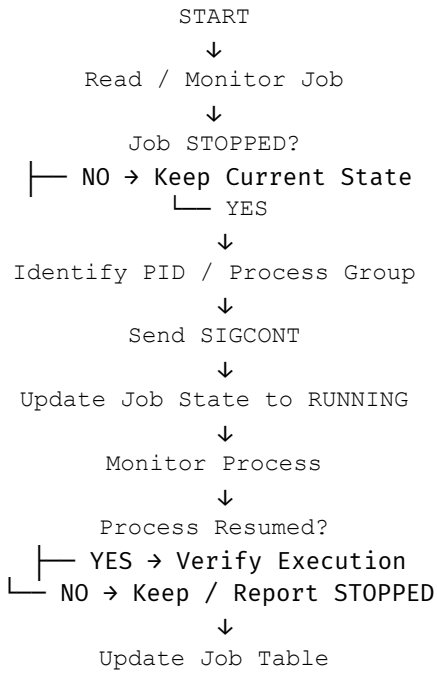
Background recovery means restoring a stopped job to active execution. The shell identifies the stopped job, sends SIGCONT, updates its status, and verifies that the process is running again.

9. Job Table

The job table stores the job ID, PID, process-group ID, command, and current state. Maintaining accurate records ensures that signal operations are applied to the correct process.

10. Control Flow

The complete control flow is kept on this page so that the flowchart is not divided across pages.



```
        ↓
Continue Monitoring
```

11. Algorithm

1. Start the program.
2. Initialize the background-job table.
3. Create a background child process.
4. Store its job ID, PID, process-group ID, command, and RUNNING state.
5. Send a stop signal to demonstrate a stopped job.
6. Use waitpid() with status options to detect the STOPPED state.
7. Update the job record to STOPPED.
8. Identify the stopped job using its job ID.
9. Send SIGCONT to the job or its process group.
10. Wait for the child status to confirm that it has continued.
11. Update the job state from STOPPED to RUNNING.
12. Continue monitoring the child process.
13. When the child completes, update the state to COMPLETED.
14. Remove the completed job from the active job table.
15. Display the verified execution result.
16. Stop the program.

12. Data Used

A job structure stores the job ID, PID, process-group ID, command text, and current state. Integer status values represent process events, while signal constants identify the required operating-system signals.

13. Operations Used

- fork() for creating a background process.
- setpgid() for process-group management.
- kill() for sending stop and continue signals.
- waitpid() for detecting process state changes.
- WUNTRACED for detecting stopped children.
- WCONTINUED for detecting resumed children.
- WNOHANG for non-blocking status checks.
- WIFSTOPPED(), WIFCONTINUED(), and WIFEXITED() for status verification.

14. Job Record Structure

```
Job ID PID PGID State Command
-----------------------------------------------
 [1] 4312 4312 STOPPED sleep 5
 [1] 4312 4312 RUNNING sleep 5
 [1] 4312 4312 COMPLETED sleep 5
```

15. Program

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>
```

```
#include <string.h>

typedef struct {
 int job_id;
 pid_t pid;
 pid_t pgid;
 char command[50];
 char state[20];
 } Job;

void print_state(Job *job) {
 printf("[%d] PID=%d PGID=%d STATE=%s CMD=%s\n",
job->job_id,
 job->pid,
 job->pgid,
 job->state,
 job->command);
 }

int main() {
 Job job;
 int status;

 printf("Skill - 18: Signal Handling and Job Recovery\n");
printf("-----------------------------------------------\n");

 pid_t pid = fork();

 if (pid < 0) {
 perror("fork");
 return 1;
 }

 if (pid == 0) {
 setpgid(0, 0);

 for (int i = 1; i <= 5; i++) {
 printf("Background job running... %d\n", i); sleep(2);
 }

 exit(0);
 }

 setpgid(pid, pid);

 job.job_id = 1;
 job.pid = pid;
 job.pgid = pid;
 strcpy(job.command, "background-task");
 strcpy(job.state, "RUNNING");

 printf("\nJob created:\n");
 print_state(&job);

 sleep(1);

 printf("\nSending SIGSTOP...\n");
 kill(-job.pgid, SIGSTOP);

 waitpid(job.pid, &status, WUNTRACED);

 if (WIFSTOPPED(status)) {
 strcpy(job.state, "STOPPED");
 printf("Job stopped by signal.\n");
 print_state(&job);
 }

 printf("\nSending SIGCONT...\n");
 kill(-job.pgid, SIGCONT);
 waitpid(job.pid, &status, WCONTINUED);

 if (WIFCONTINUED(status)) {
 strcpy(job.state, "RUNNING");
 printf("Job continued successfully.\n");
 print_state(&job);
 }

 waitpid(job.pid, &status, 0);

 if (WIFEXITED(status)) {
 strcpy(job.state, "COMPLETED");
 printf("\nBackground job completed.\n");
 print_state(&job);
```

```
    }
    printf("\nJob record removed after completion.\n");
    printf("Signal recovery test completed successfully.\n");

    return 0;
}
```

16. Program Explanation

16.1 Job Creation

The parent creates a child process and places it in its own process group. The job record stores the PID, PGID, command, and initial RUNNING state.

16.2 SIGSTOP

The parent sends SIGSTOP to the process group. The child stops, and waitpid() with WUNTRACED allows the parent to detect the stopped state.

16.3 Job Status Update

After detecting WIFSTOPPED(), the job table state changes from RUNNING to STOPPED. The updated record shows the current state.

16.4 SIGCONT

The parent sends SIGCONT to the stopped process group. WCONTINUED allows the parent to verify that the child has resumed, after which the job state is changed back to RUNNING.

16.5 Completion and Cleanup

The parent waits for the child to finish. Once WIFEXITED() is detected, the state becomes COMPLETED and the job record can be removed from the active job table.

17. Sample Output

```
Skill - 18: Signal Handling and Job Recovery
---------------------------------------------

Job created:
 [1] PID=4312 PGID=4312 STATE=RUNNING CMD=background-task

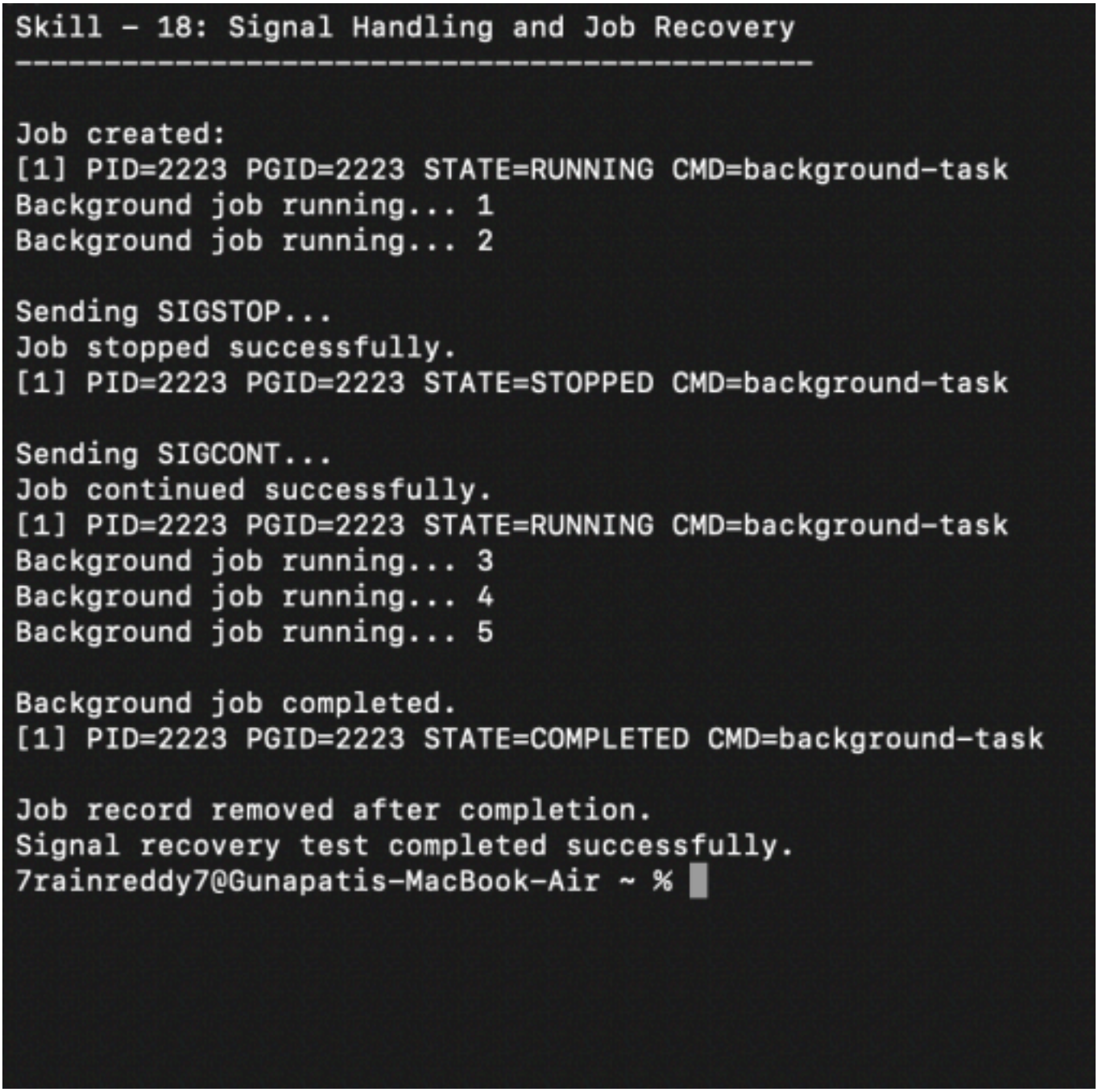
Sending SIGSTOP...
Job stopped by signal.
 [1] PID=4312 PGID=4312 STATE=STOPPED CMD=background-task

Sending SIGCONT...
Job continued successfully.
 [1] PID=4312 PGID=4312 STATE=RUNNING CMD=background-task
Background job running... 1
Background job running... 2
Background job running... 3
Background job running... 4
Background job running... 5

Background job completed.
 [1] PID=4312 PGID=4312 STATE=COMPLETED CMD=background-task

Job record removed after completion.
Signal recovery test completed successfully.
```

18. Output Screenshot



19. Testing

- Tested creation of a background job and process group.
- Tested delivery of SIGSTOP to stop the background process.
- Verified the STOPPED state using waitpid() status information.
- Tested delivery of SIGCONT to resume the stopped process.
- Verified the RUNNING state after continuation.
- Verified that the background process continued execution.
- Verified the COMPLETED state after normal termination.
- Verified removal of the completed job record.

20. Test Cases

Test Case	Input / Scenario	Expected Result
1	Create background task	Job record is created in RUNNING state.
2	Send SIGSTOP	Job changes to STOPPED state.
3	Send SIGCONT	Job resumes and changes to RUNNING state.
4	Monitor resumed task	Background task continues execution.
5	Process exits normally	Job changes to COMPLETED.
6	Cleanup job record	Completed record is removed.

2

1. Observation

- The background process was created and assigned a process group.
- SIGSTOP successfully moved the job to the STOPPED state.
- The job table recorded the stopped state.
- SIGCONT successfully resumed the process.
- The job table was updated back to RUNNING.
- The resumed process continued its normal execution.
- The process reached the COMPLETED state.
- The completed job record was removed after execution.

22. Result

Signal delivery and background-job recovery were successfully implemented. The program created a process group, stopped the background job using SIGSTOP, detected and recorded the STOPPED state, sent SIGCONT to resume execution, updated the job state to RUNNING, verified continued execution, and removed the job record after completion.

23. Conclusion

The practical demonstrated how operating systems use signals and process groups to control process execution. It also showed how a shell can recover stopped background jobs by sending SIGCONT, updating its job table, verifying resumed execution, and cleaning up completed records.